

RELAYSPEC: ADAPTING FROZEN SPECULATIVE DRAFTERS WITH LINEAR MAPS

Anonymous authors

Paper under double-blind review

ABSTRACT

As Large Language Models (LLMs) continue to advance, efficient inference remains a challenge. Speculative decoding reduces sequential generation by drafting several tokens for parallel target verification. Drafters such as EAGLE and DFlash use target hidden states, making reuse harder when the target changes. We introduce RelaySpec, which adapts a pretrained DFlash drafter by training five linear maps on its target-feature inputs while keeping the draft transformer frozen. For fine-tuned targets, we train the maps to improve token proposals using target-generated responses. For cross-model transfer, we train them to reconstruct the features expected by the drafter. The maps fold into the existing fusion projection before inference. The same construction applies to EAGLE-3 with three maps, which gain 46 to 106 percent on fine-tuned targets. We evaluate adaptation to fine-tuned targets, transfer across model sizes and families, and shared serving. A Qwen3-4B drafter transferred to Qwen3-8B reaches $5.82\times$ autoregressive throughput on MATH and $4.68\times$ on GSM8K, recovering 100.2% and 99.4% of native-drafter throughput. Fitting these maps uses 16,384 training examples, about 49 times fewer than the published recipe for training a DFlash drafter, and updates no draft weight. Transfer is weaker on code and chat. A nonlinear map with more parameters does not improve on the linear one in our benchmarks. Comparisons with draft-body adaptation and feature interventions show when adapting the drafter’s inputs is useful and where its benefits are limited.

1 INTRODUCTION

As Large Language Models (LLMs) continue to advance, efficient inference remains a challenge. Autoregressive decoding generates one token at a time. This inherent seriality creates a major performance bottleneck: decoding is slow, memory-bound, and fails to fully utilize modern GPUs (Cai et al., 2024; Vaswani et al., 2017).

Speculative decoding accelerates a language model, called the *target*, using a smaller model or prediction module, called the *drafter*. The drafter proposes several candidate tokens, and the target checks them in parallel. Verification keeps an accepted prefix and replaces the first rejected proposal with a correction. Appropriate acceptance and correction rules preserve the target’s output distribution (Leviathan et al., 2023; Chen et al., 2023). Earlier work on blockwise parallel decoding established the same basic idea of proposing several tokens and verifying them together (Stern et al., 2018), and a broad family of methods has since developed around it (Xia et al., 2024). Speedup depends primarily on the length of the accepted prefix in each proposed block, together with the time spent drafting and verifying that block.

Some drafters also use the target’s hidden-state representations. These representations provide contextual information that can help the drafter propose tokens the target will accept. Medusa adds parallel prediction heads to the target’s final hidden states (Cai et al., 2024). Other systems keep the drafter separate and instead restructure verification, using tree-based proposals (Miao et al., 2024; Chen et al., 2024), parallel Jacobi-style decoding (Fu et al., 2024), or layers of the target itself as the drafter (Zhang et al., 2024; Elhoushi et al., 2024). The EAGLE family uses target features to guide autoregressive drafting (Li et al., 2024b;c; 2025). DFlash instead uses a block-diffusion drafter to predict several tokens in one pass and injects fused target hidden-state representations into every layer of the drafter model (Chen et al., 2026).

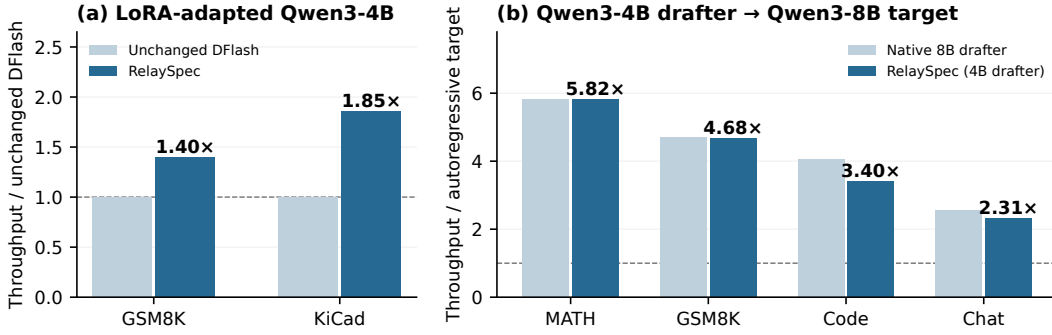


Figure 1: RelaySpec adapts a frozen drafter to a changed target. (a) On LoRA-adapted Qwen3-4B targets, the maps raise mean per-request throughput by 39.51% on GSM8K and 85.47% on KiCad over the unchanged DFlash drafter. (b) A Qwen3-4B drafter reused with Qwen3-8B reaches $5.82\times$ and $4.68\times$ autoregressive throughput on MATH and GSM8K, with the native 8B drafter shown for comparison. The panels use different baselines. Fitting the maps took under six minutes in (a) and 25 minutes in (b), and no draft weight is updated. Section 5.1 gives the evaluation settings.

This dependence complicates reuse of the drafter model when a service changes its target’s size or family, or fine-tunes it using methods such as LoRA (Hu et al., 2022). The new target can produce different hidden states and preferred continuations, and changed feature dimensions may be incompatible with the existing drafter. Training a replacement drafter model requires target-specific supervision, optimization of the drafter’s weights, and validation (Li et al., 2025; Chen et al., 2026). Repeating this process across target variants increases training and maintenance costs. Can we instead reuse an existing drafter by adapting only its target-feature inputs?

We introduce RelaySpec, a method for reusing a pretrained DFlash drafter (Chen et al., 2026) after its target changes (Figure 2). Section 2 explains the parts of DFlash that RelaySpec uses. The new target supplies hidden states from five layers. A separate linear map transforms each stream before the drafter’s original fusion projection combines them. The resulting context conditions the drafter transformer, which predicts a block of candidate tokens.

During adaptation, only the five maps are trained. The target, original fusion projection, drafter transformer, normalization layers, and the embeddings and output head selected for the model pair remain frozen. A map can be rectangular when the target and drafter expect different feature dimensions. Before inference, we multiply each map into its corresponding fusion block, a composition of linear operations. This produces one effective projection, so inference needs no separate mapping operation.

RelaySpec uses training signals that match the change in the feature interface. For a LoRA-fine-tuned target, identity-initialized maps preserve the existing interface and are trained to improve the accepted prefix. For cross-model transfer, the maps reconstruct the original target’s features and fused context from the new target’s features. Section 4 gives both procedures.

We analyze the learned maps by compressing their corrections with SVD, scaling the correction strength, comparing the corrections with LoRA-induced feature changes, and interpolating between token-trained and MSE-trained maps. We measure how these interventions change acceptance and throughput.

Our contributions are:

- We introduce RelaySpec, an adapter layer between a target model’s hidden-state representations and a pretrained DFlash drafter. It uses five trainable linear maps, keeps the drafter transformer frozen, and folds into the original fusion projection before inference.
- We evaluate adaptation across fine-tuned targets, model sizes, and families, including training-budget and shared-serving comparisons. On Qwen3-4B LoRA targets, the selected AUF maps improve mean per-request throughput over the unchanged DFlash drafter by 39.51% and 85.47% on GSM8K (Cobbe et al., 2021) and KiCad, respectively. The se-

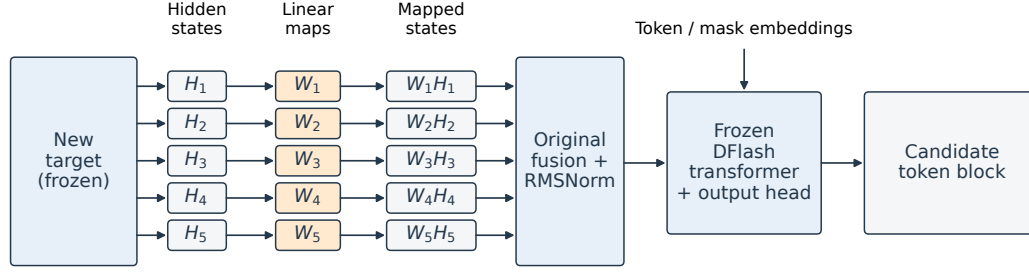
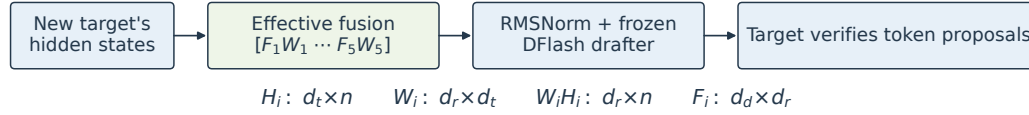
A Training: only the maps change**B Inference: maps are folded into fusion**

Figure 2: RelaySpec training and inference. Orange maps are trainable and blue model components are frozen. Each hidden-state matrix H_i enters a map W_i before fusion. A single token gives $W_i h_{i,t}$. Here n is the number of tokens, d_t is the new target’s hidden-state width, d_r is the width expected by the reused fusion projection, and d_d is the drafter transformer’s hidden width. For Qwen3-8B with the Qwen3-4B drafter, $d_t = 4096$ and $d_r = d_d = 2560$. Each W_i is 2560×4096 , each F_i is 2560×2560 , and the folded fusion projection is 2560×20480 . The new target verifies proposals after adaptation.

lected Qwen3-4B-to-8B transfer recovers 100.2% and 99.4% of native-drafter throughput on MATH and GSM8K, with weaker transfer on code and chat.

- We analyze full-map and correction-map SVD truncation, correction scaling, alignment with LoRA-induced feature changes, and interpolation toward MSE-trained maps.

2 PRELIMINARIES

2.1 DFLASH BLOCK DRAFTING

The *original target* is the model used to train the available drafter, which we call the original target’s *native drafter*. The *new target* is the model we want to accelerate.

DFlash predicts several future tokens in parallel (Chen et al., 2026). At each decoding step, it forms a block with the last token committed by the target, called the *anchor*, followed by masked proposal positions. The released Qwen3-4B DFlash checkpoint has a default block size of 16, so each block contains one anchor and 15 proposals. We keep this setting. The DFlash reference generator also defaults to temperature zero, so we use greedy decoding. One drafter pass predicts all 15 proposals, and the target checks them in parallel. With greedy verification, speculative decoding is theoretically guaranteed to produce the same token sequence as greedy target-only decoding (Leviathan et al., 2023; Chen et al., 2023).

2.2 THE TARGET-FEATURE INTERFACE

DFlash also conditions the drafter on hidden-state representations from five fixed target layers. We use d_t for the new target’s hidden-state width, d_r for the original target’s hidden-state width expected by the reused fusion projection, and d_d for the drafter transformer’s hidden width. Let $h_{i,t}^{(o)} \in \mathbb{R}^{d_r}$ be the original target’s representation at token t from layer i . The original fusion projection has five blocks $F_i \in \mathbb{R}^{d_d \times d_r}$. Their horizontal concatenation is $F = [F_1 \cdots F_5] \in \mathbb{R}^{d_d \times 5d_r}$. For one

token, DFlash computes

$$c_t^{(o)} = N \left(\sum_{i=1}^5 F_i h_{i,t}^{(o)} \right) \in \mathbb{R}^{d_d}, \quad (1)$$

where $N : \mathbb{R}^{d_d} \rightarrow \mathbb{R}^{d_d}$ is the original RMSNorm (Zhang & Sennrich, 2019). The sum is equivalent to concatenating the five representations and applying F .

DFlash supplies the fused context as key and value entries to every transformer layer of the drafter model (Chen et al., 2026; Vaswani et al., 2017). The anchor and masked positions provide the queries. Only target representations before the anchor are visible, so the drafter cannot access the tokens it is predicting.

RelaySpec adapts this interface. For n tokens, the new target supplies $H_i = [h_{i,1} \cdots h_{i,n}] \in \mathbb{R}^{d_t \times n}$ from layer i , where each $h_{i,t} \in \mathbb{R}^{d_t}$. RelaySpec learns $W_i \in \mathbb{R}^{d_r \times d_t}$. It maps one vector to $W_i h_{i,t} \in \mathbb{R}^{d_r}$ and the full sequence to $W_i H_i \in \mathbb{R}^{d_r \times n}$. The adapted context for one token is

$$\hat{c}_t = N \left(\sum_{i=1}^5 F_i W_i h_{i,t} \right) \in \mathbb{R}^{d_d}. \quad (2)$$

The maps can be folded into $G = [F_1 W_1 \cdots F_5 W_5] \in \mathbb{R}^{d_d \times 5d_t}$. The fusion projection, RMSNorm, drafter transformer, token embedding, and vocabulary head remain frozen. If the new target uses a different tokenizer, a separate vocabulary conversion is required.

3 RELATED WORK

Adapting and reusing drafters. EDA and RelaySpec both use target-specific training data, but EDA separates the drafter into shared and target-specific components and trains the lightweight private component, whereas RelaySpec trains five maps at DFlash’s hidden-state input and leaves the drafter transformer unchanged (Lin et al., 2026). Online Speculative Decoding continually updates one or more drafters from observed serving queries, and OmniDraft adds an online n-gram cache, hybrid distillation, and vocabulary conversion, whereas RelaySpec trains its maps offline and performs no serving-time training (Liu et al., 2024; Ramakrishnan et al., 2025). PARD trains a target-independent parallel drafter for a family of targets, and PARD-2 trains one drafter to support target-independent and target-conditioned modes, whereas RelaySpec starts from an already trained target-conditioned DFlash drafter and changes only its feature interface (An et al., 2026a;b).

Adapting pretrained drafters through target features. SD^2 is the closest method-level comparison because it also conditions a pretrained drafter on target hidden states (Berdoz et al., 2026). Its main method injects a learned steering signal into every feed-forward layer and fully fine-tunes the autoregressive drafter, whereas RelaySpec maps the five feature streams already used by DFlash and keeps its transformer frozen. SD^2 also tests a frozen drafter, but its signal is still injected within every feed-forward layer, whereas RelaySpec places the maps before the existing fusion projection and folds them into that projection before inference.

Other directions in speculative decoding. A parallel line of work keeps the drafter fixed and changes how proposals are structured or verified, through tree-structured candidates and verification (Miao et al., 2024; Chen et al., 2024; Cheng et al., 2024), Jacobi-style parallel decoding (Fu et al., 2024), or self-speculation from the target’s own early layers (Zhang et al., 2024; Elhoushi et al., 2024). A complementary line improves the drafter by distillation and objective design rather than by changing its interface (Zhou et al., 2024). Surveys of the area cover both (Xia et al., 2024). RelaySpec is orthogonal to all of these: it changes neither the proposal structure nor the verification rule, and it can in principle sit in front of any drafter that consumes target hidden states.

Structural re-parameterization. RepSpec adds redundant linear structure during draft training and merges it away before inference (Huo et al., 2026). RelaySpec uses the same property in a different place, training one map per feature stream and folding all of them into the existing fusion projection, so the adapted drafter has exactly the inference cost of the original. Linear maps between the representation spaces of different models have also been used to transfer steering vectors and

probes (Huang et al., 2025; Chen et al., 2025), which motivates mapping hidden states but does not establish that a mapped drafter keeps its acceptance or speed.

4 METHOD

RelaySpec reuses a pretrained speculative drafter when its target is fine-tuned or replaced by a different model. We train linear maps that transform the new target’s hidden states into inputs the existing drafter can use, while keeping the target and the original drafter weights frozen. The drafter uses these mapped features to propose tokens, and the new target verifies them. This lets us adapt the drafter to a changed target without retraining its prediction network. Our main implementation and experiments use DFlash (Chen et al., 2026), so we describe its architecture and training procedures first, then apply the same construction to EAGLE-3.

4.1 ADAPTING THE DRAFTER’S INPUTS

We retain the drafter’s K target-layer inputs and place one map before each original fusion block (Chen et al., 2026). DFlash uses $K = 5$. Map W_i connects the new target’s selected feature stream i to the corresponding original drafter input. The same map is applied at every token position. At position t , let $h_{i,t} \in \mathbb{R}^{d_i}$ be the new target’s feature from selected layer i . A bias-free map $W_i \in \mathbb{R}^{d_r \times d_i}$ converts it to the width d_r expected by the drafter. The frozen fusion block $F_i \in \mathbb{R}^{d_d \times d_r}$ projects it to the draft width d_d . The K streams give the context

$$\hat{c}_t = N\left(\sum_{i=1}^K F_i W_i h_{i,t}\right), \quad (3)$$

where N is the original RMSNorm (Zhang & Sennrich, 2019). Only $W_1, \dots, W_K$ are trained. The target, fusion projection, normalization, and draft transformer remain frozen. We also retain and freeze the original drafter’s token embeddings and vocabulary output head.

After training, we multiply each learned matrix W_i by its corresponding fusion block F_i and join the K products into one matrix:

$$G = [F_1 W_1 \dots F_K W_K] \in \mathbb{R}^{d_d \times K d_i}, \quad \hat{c}_t = N(G[h_{1,t}; \dots; h_{K,t}]). \quad (4)$$

The bracketed features are stacked into one vector. Block multiplication gives $G[h_{1,t}; \dots; h_{K,t}] = \sum_i F_i W_i h_{i,t}$, so one multiplication by G replaces the separate map and fusion operations in exact arithmetic. The original normalization remains afterward. Figure 2 shows both computations. We keep the maps linear because a linear map both changes the feature width and folds into the fusion projection. In our benchmarks a nonlinear alternative with more parameters does not improve on it, as Section 5.3.3 reports.

4.2 TRAINING FOR A LoRA-ADAPTED TARGET

Initialization. When LoRA fine-tuning (Hu et al., 2022) changes the target, its feature dimensions remain the same. We therefore initialize each map to the identity, $W_i = I$, so inserting the maps does not disrupt the existing drafter’s behavior with the adapted target. At initialization, $F_i W_i h_{i,t} = F_i h_{i,t}$ for every input, so the maps leave the original computation unchanged. This preserves the starting behavior with the active LoRA target, not the behavior of the target before fine-tuning. Training then learns changes $\Delta_i = W_i - I$ that help the frozen drafter predict the adapted target’s tokens.

Training data. We generate responses with the adapted target and save their tokens and hidden states. At sampled response positions, the drafter receives one known token followed by 15 masked positions to predict. Together they form a block of 16 tokens, matching the pretrained Qwen DFlash configuration (Chen et al., 2026). We retain this block structure to adapt the existing drafter without changing its prediction procedure. Each block can use target features only from before its starting position. Following DFlash, attention connects positions within a block but not separate blocks (Chen et al., 2026).

Objective. Because verification stops at the first rejected proposal, we supervise the correct prefix and the first incorrect prediction, excluding later positions. This is the accept-until-fail (AUF)

objective (Yang & Li, 2026). For block b and proposal position j , let q_{bj} be the predicted token distribution, y_{bj} the saved target token, and m_{bj} a mask that excludes padding and non-response positions. We compute

$$u_{bj} = \mathbf{1} \left[\arg \max_v q_{bj}(v) = y_{bj} \text{ or } m_{bj} = 0 \right], \quad s_{bj} = \text{stopgrad}(\prod_{k < j} u_{bk}),$$

$$\mathcal{L}_{\text{AUF}} = - \frac{\sum_{b,j} m_{bj} s_{bj} \log q_{bj}(y_{bj})}{\sum_{b,j} m_{bj} s_{bj}}. \quad (5)$$

The product includes only earlier predictions, so the first failure still receives supervision. We recompute this mask from current predictions and hold it fixed during differentiation. Gradients pass through the frozen drafter to update the maps. We normalize the loss within each microbatch before averaging across accumulation steps and workers.

4.3 TRAINING FOR TRANSFER ACROSS MODELS

Initialization. A drafter trained with one target expects that target’s hidden states. Replacing the target can change both their dimensions and meaning. We therefore train the maps to reconstruct features from the original target, the model used to train the drafter. For this setting, we initialize the maps with independent Xavier-uniform values (Glorot & Bengio, 2010).

Training data. We generate responses with the new target, then run both targets causally on the same saved prompt and response to collect paired hidden states. The original target does not generate a separate response. With a shared tokenizer, both receive identical token IDs. When tokenizers differ, we pair only positions corresponding to the same text prefix. The original target is needed to prepare these training features, but is not used during adapted inference.

Objective. For example b and position t , let h_{bti} and $h_{bti}^{(o)}$ denote the paired new- and original-target features in stream i . The original features supply the context we aim to reconstruct:

$$c_{bt}^{(o)} = N \left(\sum_{i=1}^K F_i h_{bti}^{(o)} \right). \quad (6)$$

For B examples, let S_b contain the selected valid prompt and response positions in example b . Using relative squared error, we minimize

$$D(u, v) = \frac{\|u - v\|_2^2}{\|v\|_2^2 + \varepsilon}, \quad \varepsilon = 10^{-6}, \quad (7)$$

$$\mathcal{L}_{\text{MSE}} = \frac{1}{B} \sum_{b=1}^B \frac{1}{|S_b|} \sum_{t \in S_b} \left[\frac{1}{K} \sum_{i=1}^K D(W_i h_{bti}, h_{bti}^{(o)}) + D(\hat{c}_{bt}, c_{bt}^{(o)}) \right]. \quad (8)$$

The first term averages reconstruction errors across the K feature streams. The second reconstructs the combined context received by the draft layers. These two terms have equal weight. We exclude padding and average positions within each example before averaging examples. Unlike AUF, this objective trains the maps without passing gradients through the draft transformer. Dividing by each original vector’s squared norm makes the error relative to its magnitude. The context term complements the layer terms because errors in different streams can interact when the fusion projection combines them. Reconstruction is a training objective, not a guarantee of accepted tokens, so we evaluate live decoding separately.

4.4 EXTENSION TO EAGLE-3

The construction is not specific to DFlash. EAGLE-3 combines features from three target layers before autoregressive drafting (Li et al., 2025), so $K = 3$. We place one map before each of those inputs and leave the original feature projection, prediction network, and autoregressive drafting procedure unchanged. The maps fold into that projection exactly as in Equation 4, so inference again runs the original architecture with one updated projection.

We pair each setting with the same objective we use for DFlash. For a LoRA-adapted target the feature widths match, so the maps start at the identity and receive token supervision. Because

EAGLE-3 drafts one token at a time instead of filling a masked block, we apply the accept-until-fail rule along its own recurrent path. At each sampled anchor the drafter predicts up to eight successive tokens, a position is supervised only when every earlier prediction was correct, and the first incorrect prediction is still supervised. A saved token continues a path only where it matches the greedy prediction. The frozen drafter carries a reduced vocabulary head, so a target token outside that head ends a path without contributing a loss term, and we normalize each example by its own count of supervised positions. For a replaced target the widths can differ, so the maps start at Xavier-uniform values and train by reconstruction using Equation 8.

4.5 GENERATION AND VERIFICATION

The new target computes hidden states during prompt processing and proposal verification. We retain states from the committed prefix and map them into context for subsequent drafting. The frozen drafter proposes tokens, and the target verifies them using its existing decoding procedure. After verification, the decoder removes rejected speculative positions from the caches before the next drafting step. The original target is no longer needed to supply features during generation. Cross-tokenizer transfer additionally requires a proposal-conversion procedure, which the hidden-state maps do not provide.

Greedy verification. For greedy decoding, changing the drafter does not change the target’s output if block verification makes the same choices as sequential target evaluation on identical prefixes. To see this, assume that the committed prefix matches autoregressive decoding. Each accepted proposal equals the target’s next choice, and the first rejected proposal is replaced by that choice. If all proposals are accepted, the target supplies the next token. The extended prefix therefore still matches, and induction establishes the result for the complete sequence (Leviathan et al., 2023; Chen et al., 2023). This argument assumes consistent token handling, stopping rules, and target evaluation.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Drafters. Every DFlash experiment starts from one pretrained checkpoint released for Qwen3-4B (Chen et al., 2026; Yang et al., 2025). It reads five target streams at zero-based layers 1, 9, 17, 25 and 33, has draft width 2,560 with 2560×2560 fusion blocks, and drafts a 16-token block from one anchor and 15 masked positions. Section 5.4 additionally uses published EAGLE-3 checkpoints, which read three streams (Li et al., 2025). No draft transformer is ever trained.

Targets. In the *fine-tuned* setting the target is a base model carrying a LoRA adapter that the released drafter has never seen (Hu et al., 2022): Qwen3-4B with a GSM8K, KiCad or NanoCoder adapter, and Qwen3-8B with a GSM8K or CodeAlpaca (Chaudhary, 2023) adapter, each merged into the frozen target before inference. In the *replaced* setting the target is a different model from the one the drafter was built for. We evaluate four such transfers, named by the target they accelerate and the model the drafter came from: Qwen3-8B from Qwen3-4B, Qwen3-4B from Qwen3-8B, Llama3.1-8B from Qwen3-4B, and Llama3.2-3B from Llama3.1-8B (Grattafiori et al., 2024). Llama3.1-8B from Qwen3-4B is the only pair whose tokenizers differ, so it also uses the proposal bridge of Section 4.5. Maps are square 2560×2560 when the target keeps the drafter’s feature width, giving 32.8M trained parameters, and 2560×4096 when accelerating Qwen3-8B, giving 52.4M.

Reference drafters. Each setting has one meaningful reference, and we name it explicitly wherever a number is relative to it. For a fine-tuned target the reference is the *unadapted drafter*, the released checkpoint run against the fine-tuned target with no adaptation. It is the honest reference because no drafter exists for a LoRA variant, so this is what a deployment would otherwise use. For a replaced target the reference is the *native drafter*, a drafter trained specifically for the target being accelerated. This is the demanding reference, because matching it means reaching the quality of target-specific drafter training without doing any. Autoregressive decoding, the target with no drafter at all, is the common denominator for both.

Datasets and benchmarks. Transfer experiments use MATH test (Hendrycks et al., 2021), GSM8K test (Cobbe et al., 2021), pinned LiveCodeBench releases (Jain et al., 2024) and single-turn instruc-

tions derived from UltraFeedback (Cui et al., 2024), reported as Math, GSM8K, Code and Chat. Fine-tuned-target experiments evaluate on held-out prompts from the adapter’s own domain. Every reported cell uses 128 held-out prompts with a 2,048-token response cap, except KiCad at 8,192.

Metrics. RelaySpec changes only the drafter’s inputs and leaves the target and its acceptance rule untouched, so we report acceleration rather than task quality. *Speedup* is mean per-request throughput divided by that of autoregressive decoding of the same target on the same prompts, so autoregressive decoding is $1.00\times$ by definition. *Acceptance length* τ is pooled emitted tokens per verification step and includes the correction token, so its floor is one. Section 5.5 additionally reports *aggregate throughput*, total returned tokens divided by workload wall time including drain. These statistics weight requests differently and we never mix them inside one comparison. Where an ablation must choose one configuration, we select by the harmonic mean of its per-workload speedups rather than their average, because the harmonic mean of ratios is pulled down by the smallest entry and so will not let a configuration hide one weak workload behind a favourable average. We use it only for those selections.

Implementation details. For a fine-tuned target we generate 4,096 responses with that target, cache its hidden states, sample 32 anchors per example, and train one epoch, which is 512 optimizer updates at global batch 8 with AdamW (Loshchilov & Hutter, 2019) at learning rate 10^{-4} . For a replaced target we reuse 16,384 saved NuminaMath (Li et al., 2024a) trajectories, supervise a fixed 25% of prompt and response positions, and train three epochs, which is 6,144 updates at learning rate 10^{-3} . Both use seed 42 on two L40S GPUs with FP32 trainable maps over a frozen BF16 model, and both export the folded projection in BF16. Single-request measurements use the Transformers library (Wolf et al., 2020) on one L40S with greedy non-thinking decoding, CUDA-synchronized timing and warmup excluded. Serving measurements use vLLM 0.28.0 (Kwon et al., 2023) with batch-invariant operations and FlashAttention kernels (Dao, 2023). We report vLLM numbers throughout; other serving stacks such as SGLang (Zheng et al., 2024) implement speculative decoding differently and are not directly comparable. In the selected single-request comparisons the unadapted, native and adapted drafters returned identical output sequences on all 128 prompts per workload, so those ratios are not confounded by output length. Under vLLM the sequences can diverge and we treat those numbers as descriptive.

5.2 MAIN RESULTS

Table 1 reports every target, workload and reference in one place. Two conclusions follow from it.

Adapting a fine-tuned target’s features recovers a large share of lost drafting quality. Training only the five maps improves every adapter at both target scales, from +6% on CodeAlpaca to +85% on KiCad, and raises acceptance length in every cell. The size of the gain tracks acceptance rather than any change to the inference path: KiCad nearly doubles acceptance length, from 5.99 to 11.26, and reaches $8.13\times$ autoregressive throughput, whereas CodeAlpaca moves from 5.62 to 5.63 and gains little. The improvement is therefore a function of how far the adapter moved the target away from what the drafter expects, which is exactly the quantity the method is designed to absorb.

A drafter can be moved to a different target model and still approach one built for it. Every transfer accelerates its target on every workload, so a frozen draft transformer can be driven by mapped features from a model it was never trained on, including across a family and a tokenizer boundary. Within the Qwen3 family the maps reach $1.002\times$ and $0.994\times$ of a native drafter on Math and GSM8K when moving up in scale, without any target-specific drafter training. Recovery is lower on Code and Chat, and lower again across families, so we report the setting as effective within a family and partial across one.

Table 2 reports a fifth transfer, a Llama3.1-8B drafter reused for Llama3.2-3B. Four trained input maps reach $2.35\times$ autoregressive throughput on Math and above $1.6\times$ on every workload.

Table 1: Complete single-request results. AR is the measured autoregressive tokens per second for that target on that workload, the common $1.00\times$ baseline for every speedup in the row. Speedup is mean per-request throughput divided by AR. τ is average acceptance length including the correction token. The reference for a fine-tuned target is the released drafter used with no adaptation, so $\times\text{ref}$ above 1 is an improvement over what a deployment would otherwise run. The reference for a replaced target is a drafter trained for that target, so $\times\text{ref}$ near 1 means RelaySpec matches target-specific drafter training without performing any. A target written $A \leftarrow B$ is target A accelerated by a drafter built for B . Bold marks $\times\text{ref}$ at or above parity.

Target $\leftarrow$ drafter	Workload	AR	RelaySpec		Reference drafter		$\times$ ref
			Speedup	τ	Speedup	τ	
<i>Fine-tuned target. Reference is the released drafter, used unadapted.</i>							
Qwen3-4B + GSM8K LoRA	GSM8K	38.81	4.63 $\times$	5.99	3.32 $\times$	4.47	1.40
Qwen3-4B + KiCad LoRA	KiCad	38.42	8.13 $\times$	11.26	4.39 $\times$	5.99	1.85
Qwen3-4B + NanoCoder LoRA	NanoCoder	39.01	4.40 $\times$	5.55	3.87 $\times$	5.11	1.14
Qwen3-8B + GSM8K LoRA	GSM8K	37.89	4.59 $\times$	6.35	3.40 $\times$	4.76	1.35
Qwen3-8B + CodeAlpaca LoRA	CodeAlpaca	37.56	4.32 $\times$	5.63	4.07 $\times$	5.62	1.06
<i>Replaced target. Reference is a drafter trained for the target being accelerated.</i>							
Qwen3-8B $\leftarrow$ Qwen3-4B	Math	37.84	5.82 $\times$	7.98	5.81 $\times$	8.21	1.00
	GSM8K	37.76	4.68 $\times$	6.24	4.71 $\times$	6.48	0.99
	Code	37.50	3.40 $\times$	4.74	4.04 $\times$	5.64	0.84
	Chat	35.09	2.31 $\times$	2.89	2.55 $\times$	3.32	0.90
Qwen3-4B $\leftarrow$ Qwen3-8B	Math	39.42	5.49 $\times$	7.66	5.70 $\times$	8.09	0.96
	GSM8K	39.72	4.27 $\times$	5.99	4.46 $\times$	6.34	0.96
	Code	39.60	3.30 $\times$	4.89	4.34 $\times$	6.26	0.76
	Chat	40.48	2.01 $\times$	2.83	2.36 $\times$	3.36	0.85
Llama3.1-8B $\leftarrow$ Qwen3-4B	Math	41.00	2.97 $\times$	4.63	3.17 $\times$	5.12	0.94
	GSM8K	41.11	2.21 $\times$	3.23	3.10 $\times$	4.42	0.71
	Code	40.79	1.98 $\times$	3.05	3.00 $\times$	4.52	0.66
	Chat	40.96	1.48 $\times$	2.06	2.79 $\times$	3.95	0.53

Table 2: Llama3.1-8B drafter reused for Llama3.2-3B. Speedup is relative to autoregressive decoding (AR), the same $1.00\times$ baseline convention as Table 1.

Workload	AR (tok/s)	Speedup	τ
Math	61.90	$2.35\times$	4.78
GSM8K	64.45	$2.18\times$	4.16
Code	63.09	$1.88\times$	3.44
Chat	64.73	$1.61\times$	2.97

5.3 ABLATION STUDIES

We ablate the three decisions that define the method: where the trained parameters sit, which objective trains them, and how much capacity and supervision they need. Each ablation holds data, schedule and evaluation fixed and changes one element.

5.3.1 WHERE THE TRAINED PARAMETERS SIT

RelaySpec trains one map per feature stream and then folds those maps into the fusion block in front of which they sit. Because the folded result is a single matrix, the natural objection is that the factorization is irrelevant and any trainable matrix of the same size would do. Table 3 tests that, against four alternatives:

- **Single fused projection.** One matrix of the same total size, trained in place of the five maps and initialized to exactly the function the five maps start from. This is the decisive control, since it differs from RelaySpec only in being unfactorized.

Table 3: Where the trained parameters sit, for fine-tuned Qwen3-4B targets under a matched accept-until-fail objective, 4,096 examples and 32 anchors. Values are percent gain in mean throughput over the unadapted drafter. HM is the harmonic mean of the corresponding speedup ratios and is the selection criterion. Best in each column in bold.

Trained object	Parameters	GSM8K	KiCad	NanoCoder	HM
Five per-stream maps (RelaySpec)	32.8M	+37.6	+89.5	+8.1	1.376
Single fused projection	32.8M	+32.4	+44.5	+8.2	1.265
Draft-transformer LoRA, rank 128	36.7M	+36.4	+75.5	+9.8	1.355
Single joint map over all streams	163.8M	+24.3	+88.4	-7.0	1.244
Rank-128 maps	3.3M	+3.2	+58.4	-28.8	0.999
Rank-56 maps	1.4M	-21.9	+35.1	-49.3	0.751
Rank-28 maps	0.7M	-43.1	+12.8	-62.5	0.565

- **Draft-transformer LoRA.** A rank-128 LoRA on the attention and MLP projections inside the draft blocks. This trains the drafter’s body instead of its inputs, and is the established alternative that RelaySpec is meant to replace.
- **Single joint map.** One dense matrix over all five concatenated streams, which permits mixing between streams at five times the parameters.
- **Rank- r maps.** Each whole map is replaced by a rank- r product, rather than adding a low-rank correction to a retained full map.

What this shows. For scale, the unadapted drafter used as the 0% reference here already reaches $3.3\times$ to $4.4\times$ autoregressive decoding before any adaptation (Table 1), so every percentage below is the further gain adaptation adds on top of that autoregressive speedup, not the whole story. The factorization does the work, not the parameter count. With identical parameters and an identical starting function, the single fused projection reaches a harmonic mean of 1.265 against 1.376 for the five maps and loses 45 points of gain on KiCad. The same contrast appears when transferring to Qwen3-8B, where under a matched objective the single fused projection reaches $0.365\times$ the native drafter on Math while the five maps reach $0.884\times$. Keeping one map per stream therefore changes what the optimizer can reach, even though the two parameterizations describe the same functions after folding and export as the same matrix. This is the behaviour that structural re-parameterization predicts, where a training-time factorization that merges away at inference still improves the trained result (Huo et al., 2026). Replacing each whole map by a low-rank product degrades monotonically, and the two smallest ranks fall below the unadapted drafter, because replacing the map discards its ability to pass the original feature through rather than merely limiting the learned correction. Draft-transformer LoRA is competitive and wins on NanoCoder, so we do not claim that adapting inputs beats adapting the body on every workload. Its cost is that the draft transformer is no longer shared, which Section 5.5 shows is what makes the method useful in deployment.

5.3.2 WHICH OBJECTIVE TRAINS THE MAPS

The two settings start from different places. With a fine-tuned target the maps start at the identity and the drafter already reads a usable interface, so what remains is to predict a shifted token distribution. With a replaced target the maps start random and possibly rectangular, so the interface itself is not yet usable. Table 4 compares the candidates and selects by harmonic mean.

What this shows. The harmonic mean makes the selection unambiguous and shows why an average would not. For a fine-tuned target the two token objectives are nearly tied on the arithmetic mean, 1.451 for accept-until-fail against 1.445 for decaying cross-entropy, because the latter’s strong KiCad result offsets its weak NanoCoder result. The harmonic mean separates them, 1.376 against 1.353, because it refuses to let the 1.031 NanoCoder entry be averaged away, and a deployment meets that workload. Reconstruction on the same fine-tuned targets reaches only 1.052, behind both token objectives and, on NanoCoder, behind the unadapted drafter itself, because it trains the maps toward the base model’s features rather than toward the tokens the adapted target actually produces. We therefore select accept-until-fail for fine-tuned targets. For a replaced target the ordering reverses and becomes decisive: both reconstruction variants exceed 0.918 while the best token objective reaches only 0.623, and the gap comes almost entirely from Code and Chat, where token supervision collapses to 0.406 and 0.573. We select reconstruction there. Among the two token

Table 4: Which objective trains the maps. The upper block reports throughput relative to the unadapted drafter for fine-tuned Qwen3-4B targets. The lower block reports throughput relative to the native Qwen3-8B drafter for transfer from Qwen3-4B, on the matched 8,192-example fits. MSE50 and MSE100 supervise that fraction of prompt and response positions. Best in each column in bold. HM is the selection criterion.

Setting	Objective	Workload			HM	
		GSM8K	KiCad	NanoCoder		
Fine-tuned	Accept-until-fail	1.376	1.895	1.081	1.376	
	Decaying cross-entropy	1.368	1.935	1.031	1.353	
	Reconstruction, MSE50	1.011	1.191	0.977	1.052	
Replaced		Math	GSM8K	Code	Chat	
	Reconstruction, MSE100	0.998	0.990	0.828	0.891	0.921
	Reconstruction, MSE50	0.999	0.975	0.832	0.888	0.919
	Decaying cross-entropy	0.886	0.919	0.406	0.573	0.623
	Accept-until-fail	0.884	0.911	0.385	0.558	0.604

objectives the harmonic mean prefers decaying cross-entropy for a replaced target, the reverse of the fine-tuned ordering, so neither token objective dominates across settings and the pairing of objective to setting is doing real work in both directions. Doubling supervised positions from MSE50 to MSE100 moves the harmonic mean by 0.002 while tripling the fitting time, so the deployed recipe supervises a quarter of positions. One qualification: the reconstruction runs use learning rate 10^{-3} and the token runs 10^{-4} , so these compare complete recipes rather than losses at one fixed setting.

Two further checks support this choice without changing it. First, on a separate matched architecture (a single low-rank fusion update rather than the five per-stream maps, held fixed to isolate the loss), plain uniform cross-entropy, the DFlash decaying weighting, and accept-until-fail were compared under identical data and schedule on NanoCoder: accept-until-fail reaches $1.072\times$ autoregressive decoding against $1.046\times$ for uniform cross-entropy and $1.047\times$ for decaying cross-entropy, a 2.4% gain over uniform weighting that decaying weighting does not reproduce. This confirms that the prefix-aware accept-until-fail weighting, not merely having some position-dependent weighting, is what improves throughput. Second, for a fine-tuned target the training responses do not need to come from the fine-tuned target itself: fitting the same architecture from the base model’s own generations reaches $1.001\times$ the throughput of fitting from the fine-tuned target’s generations on the fine-tuned target’s evaluation prompts, with a paired 95% interval of $[0.995, 1.007]$ that contains 1. The gain therefore reflects adaptation to the workload distribution rather than a fit to the fine-tuned target’s specific outputs, which is why we generate training responses with whichever target will be evaluated rather than requiring matched labeled data.

5.3.3 WHETHER THE MAPS NEED TO BE NONLINEAR

The maps are linear because a linear map changes the feature width and still folds into the fusion projection. To test whether that costs accuracy, we add a parallel nonlinear branch to each map, so stream i contributes $W_i h_i + B_i \text{SiLU}(A_i h_i)$ (Elfwing et al., 2018), with A_i reducing to 256 coordinates and B_i initialized to zero. Training therefore starts at exactly the linear solution and departs from it only if that lowers the loss. The branch raises trained parameters from 32.8M to 39.3M for a fine-tuned target and from 52.4M to 60.9M for transfer, and unlike the linear map it cannot be folded away before inference. Table 5 holds data, objective and schedule fixed within each setting.

What this shows. The nonlinear branch is given every advantage, since it starts from the linear solution, carries more parameters and is selected by the optimizer only if it lowers the loss. It nevertheless changes throughput by between -2.07% and $+2.97\%$, leaves acceptance length almost unchanged, and is behind the linear map on five of the seven workloads. The linear form is therefore not a concession made for foldability. It is at least as accurate here and additionally collapses to one fixed matrix before inference. With differences of this size and one timed pass per cell we claim only that this nonlinear interface does not pay for itself.

Table 5: Whether the maps need to be nonlinear. Values are mean tokens per second and acceptance length under matched data, objective and schedule. The nonlinear alternative carries 16 to 20 percent more parameters and remains an active computation at inference. Best in each row in bold.

Setting	Workload	Tokens/s			τ	
		Linear	Nonlinear	Change	Linear	Nonlinear
Fine-tuned	GSM8K	179.83	177.94	-1.05%	5.988	5.953
	KiCad	312.47	321.76	+2.97%	11.256	11.411
	NanoCoder	171.73	169.74	-1.16%	5.552	5.520
Replaced	Math	220.26	215.71	-2.07%	7.984	7.980
	GSM8K	176.87	174.56	-1.31%	6.242	6.257
	Code	127.61	128.41	+0.63%	4.739	4.782
	Chat	80.93	80.78	-0.20%	2.887	2.902

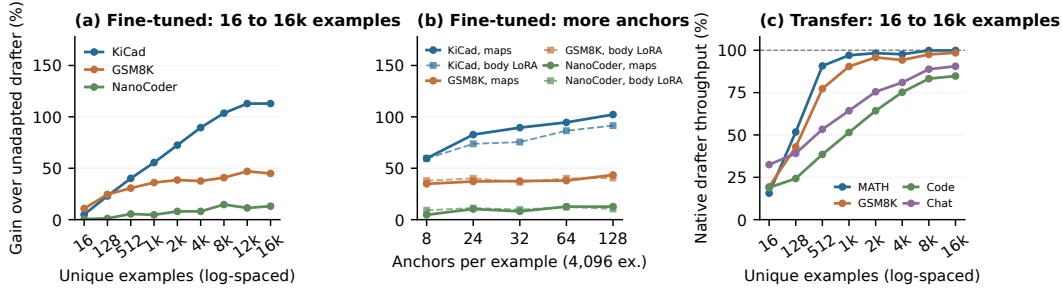


Figure 3: Supervision budget in both settings, from 16 to 16,384 training examples. (a) and (c) share a log-spaced example axis; (b) fixes examples at 4,096 and varies anchors per example. (a) and (b) use fine-tuned Qwen3-4B targets with the accept-until-fail objective, where solid lines train the five interface maps and dashed lines train a rank-128 LoRA inside the draft transformer. (c) transfers the Qwen3-4B drafter to Qwen3-8B with the reconstruction objective and plots throughput as a percentage of the native Qwen3-8B drafter. Adaptation is already effective at 16 examples and speedup keeps improving with supervision without saturating at the largest budget measured.

5.3.4 HOW MUCH SUPERVISION THE MAPS NEED

Both recipes use small budgets, which raises three questions: how little data the maps actually need, whether the larger budgets are saturated, and whether draft-transformer LoRA would overtake the maps given more data. Figure 3 varies the supervision axes for both settings from 16 to 16,384 examples, spanning three orders of magnitude.

What this shows. Adaptation is effective from as few as 16 examples. On a fine-tuned target with only 16 examples the maps already recover part of the gap, and throughput then rises smoothly and without saturating out to 16,384 examples: GSM8K goes from +10.8% to +45.0%, KiCad from +5.0% to +113.0%, and NanoCoder from +0.5% to +13.1%. Recovery of the native drafter under transfer follows the same pattern, rising from 15.6% to 99.9% on Math, 19.4% to 98.5% on GSM8K, 18.9% to 84.8% on Code, and 32.5% to 90.5% on Chat, so the workloads that transfer least well at large budgets are also the ones that still have the most to gain from more data. Because neither curve has saturated at the largest budget measured, the main results in Table 1 are conservative operating points rather than ceilings. Panel (b) shows the two curves crossing on the anchor axis: at the smallest anchor budget the maps lead draft-transformer LoRA clearly on KiCad, +59.7% against +75.5% in the opposite direction, and at 128 anchors the two are within a few points of each other on every domain. Interface adaptation is therefore the more data-efficient choice at the low end of the supervision budget, and its advantage is largest in exactly the regime the method targets, where a new target variant must be supported from very little data.

Table 6: Generalization to EAGLE-3, using three input maps. Token supervision is used for a fine-tuned target and reconstruction for a replaced one, following the selection in Section 5.3.2. The reference is the unadapted EAGLE-3 drafter for the fine-tuned block and a published EAGLE-3 drafter native to Qwen3-8B for the replaced block. Best in each row in bold. Acceptance uses EAGLE-3’s upstream counters and is not comparable with the DFlash tables.

Setting	Workload	Tokens/s			τ	
		Reference	RelaySpec	$\times$ ref	Reference	RelaySpec
Fine-tuned Qwen3-4B, token supervision	GSM8K	71.19	120.35	1.69	3.347	5.695
	NanoCoder	81.77	119.30	1.46	3.852	5.668
	KiCad	73.82	151.86	2.06	3.794	7.363
Qwen3-8B from Qwen3-4B, reconstruction	Math	72.18	77.99	1.08	3.627	3.638
	GSM8K	75.58	80.59	1.07	3.713	3.840
	Code	63.88	65.53	1.03	3.351	3.293
	Chat	61.82	63.41	1.03	3.186	3.136

5.4 GENERALIZATION TO A SECOND DRAFTER FAMILY

The construction should not depend on DFlash. We repeat both settings on published EAGLE-3 checkpoints, which read three target streams and draft one token at a time rather than filling a masked block, pairing each setting with the objective that Section 5.3.2 selected for it. Table 6 reports the result.

What this shows. Adapting the inputs of a frozen EAGLE-3 drafter improves it in both settings, by 45.9% to 105.7% on fine-tuned targets and above a purpose-built Qwen3-8B EAGLE-3 drafter on all four transfer workloads. Against autoregressive decoding of the same fine-tuned target, RelaySpec reaches $3.82\times$ on GSM8K, $3.75\times$ on NanoCoder and $5.00\times$ on KiCad, against $2.26\times$, $2.57\times$ and $2.43\times$ for the unadapted EAGLE-3 drafter. The objective selection also replicates, which is the stronger result. Training the same three maps by reconstruction on the fine-tuned targets, so that they reproduce the base model’s features instead of predicting the adapted target’s tokens, gives 71.89, 75.34 and 78.70 tokens per second, a 7.9% regression on NanoCoder and almost no change on GSM8K. Token supervision instead gives 120.35, 119.30 and 151.86 on the same targets and prompts, between 58% and 93% faster. Pairing the objective to the setting is therefore not a DFlash artifact: it holds on a different draft architecture with a different drafting procedure. Two qualifications apply. Our recurrent token objective adapts the accept-until-fail rule to a sequential drafter and is not the masked-block implementation used for DFlash, so this transfers the rule rather than the code. And unlike the DFlash comparisons the EAGLE-3 runs do not return identical sequences, so we report throughput and acceptance rather than summed latency.

5.5 SERVING MANY TARGETS FROM ONE DRAFTER

Because the draft transformer is never trained, several adapted targets can share one copy of it, with only a small matrix differing between them. This is what draft-transformer LoRA gives up. Figure 4 measures the deployment on a vLLM server holding one target backbone and one draft backbone with three LoRA targets and frozen route assignments.

What this shows. The single-request gains survive batching. Aggregate server throughput is higher for the adapted drafter at every concurrency measured, reaching 4,324 tokens per second against 2,440 for the unadapted drafter and 828 for autoregressive decoding at 32 clients, while acceptance length stays near 10.5 against 5.8. Panel (b) shows why both conventions must be reported: mean per-request throughput falls with load for all three methods even as server throughput rises, so a per-request number cannot stand in for a serving number. One pooled set of maps trained on the union of three domains reaches 98.9% to 99.6% of three separately trained sets across two repeats, so a deployment need not even hold one adapter per target. This is the practical form of the reuse question that motivates target-independent drafting (An et al., 2026b). The advantage is not unbounded: on a Qwen3-8B server the adapted drafter leads at 1, 4 and 8 clients but falls within 4% of the native drafter at 16 and 32 clients, because at high concurrency the bottleneck moves away from acceptance length.

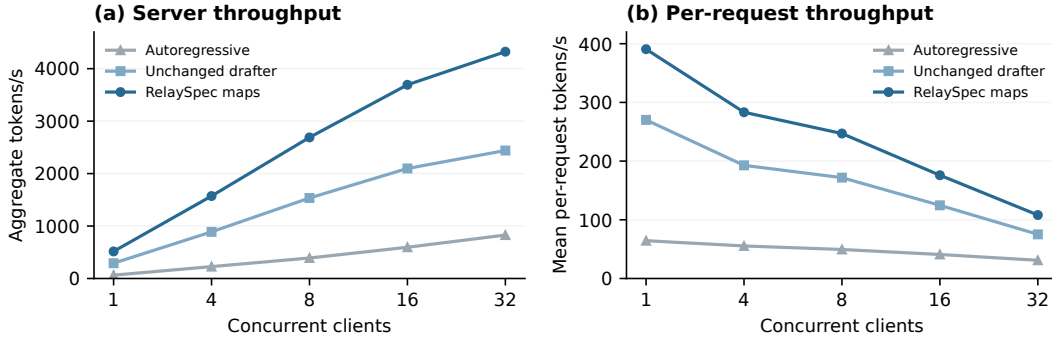


Figure 4: One Qwen3-4B target backbone and one draft backbone serving three LoRA targets under vLLM on a single L40S, 384 requests with frozen route assignments. (a) aggregate throughput, total returned tokens divided by workload wall time including drain. (b) mean per-request throughput. The adapted drafter leads on server throughput at every concurrency, while per-request throughput falls with load for all three methods.

Table 7: Aggregate vLLM throughput at 16 concurrent clients, total returned tokens divided by workload wall time including drain. Bold marks the higher of native and mapped.

Target ← drafter	Workload	Native (tok/s)	Mapped (tok/s)	Mapped/native
Qwen3-8B ← Qwen3-4B	Math	2524.31	2716.26	1.076
	GSM8K	2207.43	2262.03	1.025
	Code	1748.19	1548.47	0.886
	Chat	1091.14	1065.93	0.977
Qwen3-4B ← Qwen3-8B	Math	4482.42	3710.76	0.828
	GSM8K	3160.99	3078.87	0.974
	Code	2840.15	2002.79	0.705
	Chat	1888.69	1431.21	0.758
Llama3.1-8B ← Qwen3-4B	Math	1894.99	1046.94	0.553
	GSM8K	1354.50	853.13	0.630
	Code	1296.83	636.97	0.491
	Chat	1440.04	524.40	0.364

Concurrent serving across transfers. The same question arises for a replaced target: does the transferred drafter still help once requests are batched under vLLM? Table 7 reports aggregate throughput at 16 concurrent clients for the three transfers with a published native drafter. Batching reproduces the single-request ordering of Table 1 rather than changing it. On the transfer with the strongest single-request recovery, Qwen3-8B accelerated by the Qwen3-4B drafter, the mapped drafter leads on Math and GSM8K and trails narrowly on Code and Chat. On the reverse and cross-family transfers, where single-request recovery was already weaker, the mapped drafter trails the native drafter under concurrency on every workload, most on the cross-family Llama3.1-8B transfer. On the Llama3.2-3B transfer of Table 2, the same server configuration reaches $2.53\times$, $3.08\times$, $1.71\times$ and $1.77\times$ autoregressive throughput on Math, GSM8K, Code and Chat at 16 clients.

Memory and GPU usage. We measured post-request allocated and reserved GPU memory on the same T1 serving cells above. Allocated memory rises from 30.5914 GiB for the native drafter to 30.9190 GiB for the mapped drafter, a constant 0.33 GiB increase regardless of workload or concurrency; reserved memory shows the same small, workload-independent gap. This matches expectation: the trained maps add 52.4 million BF16 parameters to an 8-billion-parameter target and a several-billion-parameter draft transformer, so the memory cost of adaptation is negligible relative to the models it accelerates, and it does not grow with client count.

Table 8: Public-runtime comparison on 128 MATH development questions, 2,048-token cap. Speedup is each system’s own end-to-end throughput divided by its own autoregressive throughput, with a paired 95% bootstrap interval. This uses the earlier single-projection interface, not the five-map DFlash construction evaluated elsewhere in this section, and different runtimes and inherited base models prevent attributing the ranking to the adaptation method alone. Bold marks the highest observed speedup.

System	End-to-end (tok/s)	Own AR (tok/s)	Speedup [95% CI]
Single-projection interface	193.68	37.72	5.14 [4.90, 5.37]
PARD	105.63	31.67	3.34 [3.22, 3.45]
SD ² (frozen drafter)	19.92	12.48	1.60 [1.56, 1.63]

Table 9: Training examples required, against the published DFlash recipe of about 800,000 samples (Chen et al., 2026). Every fit here runs on two L40S GPUs in under half an hour.

Setting	Training examples	Reduction vs. published DFlash
Fine-tuned target (DFlash, five maps)	4,096	195× fewer
Replaced target (DFlash, five maps)	16,384	49× fewer
Fine-tuned target (EAGLE-3, three maps)	4,096	195× fewer
Replaced target (EAGLE-3, three maps)	16,384	49× fewer

5.6 COMPARISON WITH PUBLISHED SYSTEMS

The reports also include a public-runtime comparison of an earlier single-projection version of this interface against two published drafters that reuse or steer a fixed prediction network: PARD, which reuses a released parallel drafter with no new target-specific fitting (An et al., 2026a), and SD², evaluated in its frozen-drafter configuration, which steers a pretrained autoregressive drafter through injected features (Berdoz et al., 2026). All three ran on the same 128 MATH development questions with a 2,048-token cap, each reported against its own runtime’s autoregressive baseline because the systems differ in precision, attention backend and supporting libraries. Table 8 reports the result.

5.7 ADAPTATION COST

Reusing a drafter is only worthwhile if adapting it costs much less than training one. We cannot compare the two in GPU-hours from published numbers, because the DFlash paper reports its training data and schedule but not its training time or hardware budget, and it trains on different accelerators than we use (Chen et al., 2026). We therefore compare how much supervision each recipe consumes and report measured time only for our own fits, keeping fitting separate from data preparation as RepSpec does (Huo et al., 2026).

The published DFlash recipe trains all five draft layers on about 800,000 samples for six epochs, sampling 512 anchor positions per sequence at every epoch (Chen et al., 2026). Table 9 reports how many examples RelaySpec needs for each setting and the resulting reduction against that published recipe.

What this shows. Adaptation is cheap enough that supporting a new target variant is a routine operation rather than a training project: two orders of magnitude less data than training a drafter from scratch, and every fit above completes in under half an hour on two GPUs. Reconstruction is additionally the cheaper objective per example because it sends no gradient through the draft body, so the transfer fits are faster per example than the token-supervised fits despite using four times as many examples. Preparation is the honest counterweight and is not zero: reconstruction needs features from both the new and the original target on the same tokens, while token supervision needs features from the adapted target only, and generating the training responses is a further reusable cost we do not charge per fit. The comparison against published drafter training therefore holds on supervision consumed, and we do not extend it to an end-to-end wall-clock ratio.

6 ANALYSIS

We analyze how the learned maps change the hidden-state context passed to the frozen drafter.

6.1 WHAT CHANGES WHEN THE INTERFACE ALREADY WORKS?

For a LoRA-fine-tuned target, identity maps recover the native DFlash interface. Write $z_0 = \sum_i F_i h_i$ for this context and $\delta = \sum_i F_i (W_i - I) h_i$ for the learned correction. We run decoding with z_0 plus either the component of δ parallel to z_0 or the component perpendicular to it.

Table 10: LoRA-target direction and geometry controls. Acceptance length τ is measured in tokens per verification step. Bold marks the best acceptance length for each task, and underline marks the second best. The geometry measurements use 1,024 positions per task and are dimensionless.

Direction intervention			LoRA-shift geometry			
Context	GSM8K τ	NanoCoder τ	Task	Correction cosine	Shuffled cosine	Radial energy
Native z_0	4.472	5.111				
Full $z_0 + \delta$	5.988	5.552	GSM8K	-0.0429	-0.0413	25.22%
Parallel only	4.466	5.114	NanoCoder	0.1248	0.1052	34.82%
Perpendicular only	<u>5.924</u>	<u>5.527</u>				17.55%
						14.88%

The parallel component returns acceptance length to the native level, while the perpendicular component recovers nearly all of the full map’s acceptance length (Table 10). Scaling the whole context by two leaves acceptance length unchanged at 5.988 tokens per verification step on GSM8K and 5.552 on NanoCoder. The useful change is mainly directional, consistent with RMSNorm suppressing common rescaling. Appendix A.1 reports every result.

Do the maps reverse the change caused by LoRA fine-tuning? We run the base and LoRA-fine-tuned targets on the same 1,024 saved token positions. At each position, we compare the map correction with the change in fused context caused by LoRA. Exact cancellation would require the correction to point in the opposite direction. We also shuffle the LoRA changes across positions while leaving the corrections fixed. This tests whether each correction is specifically aligned with the change at the same token. The mean correction cosine is -0.0429 on GSM8K and 0.1248 on NanoCoder, close to the shuffled values of -0.0413 and 0.1052 (Table 10). The maps therefore do not systematically reverse the LoRA-induced change token by token. These results support redirection of an already usable context, not literal recovery of the pre-LoRA context. Appendix A.2 defines each measurement and gives the full procedure.

Identity-preserving SVD retains most of the benefit at moderate rank. Rank 128 on GSM8K keeps an acceptance length of 5.928 tokens per verification step, compared with 5.988 for the full correction, and retains 95.1–99.6% of its latency reduction in two repeats. Rank 256 on NanoCoder keeps 5.521 of 5.552 tokens per verification step and retains 57.8–89.3% of the latency reduction. Truncating the entire map to rank 1 or 2 instead removes the identity path and collapses drafting performance. Appendix Tables 29 and 30 report all ranks and timing repeats.

6.2 WHY DOES RECONSTRUCTION HELP ACROSS MODEL SIZES?

We study a Qwen3-8B target with a pretrained Qwen3-4B drafter. The target supplies five feature vectors with 4,096 coordinates. Five learned 2560×4096 maps convert them to the width expected by the drafter. Its fusion projection, normalization, and transformer remain frozen. MSE reconstructs the corresponding Qwen3-4B features, while AUF and decay CE train through token predictions. All three start from the same Xavier initialization. We ask whether reconstruction recovers the interface used by the frozen drafter.

Does MSE recover the Qwen3-4B interface? Yes, on the measured training positions. Context error measures the normalized squared difference from the Qwen3-4B context after fusion and RMSNorm. Source cosine measures direction agreement for each feature stream. MSE has lower error and higher cosine than AUF or decay CE (Table 11). Since MSE directly optimizes reconstruction, we next test prediction. Appendix Tables 16 and 17 report every stream.

Does this help the frozen drafter predict? Yes, most clearly on Code and Chat. At each held-out position, the frozen drafter receives the same preceding tokens and target features, then predicts the same saved continuation. Only the maps differ. Cross-entropy measures the probability assigned to the correct tokens. Correct prefix counts consecutive correct predictions before the first error. MSE gives the lowest cross-entropy and longest correct prefix on MATH, Code, and Chat. GSM8K is mixed, with decay CE giving the lowest cross-entropy and AUF the longest correct prefix. The Code and Chat results show that recovery improves prediction, not only reconstruction. Appendix Tables 18–21 report the results for every map.

Table 11: Interface recovery on 4,096 response positions from 64 NuminaMath-derived training examples. Source cosine is reported as the minimum–maximum across the five feature streams. Error and cosine are dimensionless.

Recipe	Peak LR	Error ↓	Source cosine min–max ↑
MSE	10^{-3}	0.136	0.944–0.985
AUF	10^{-4}	0.940	0.121–0.267
Decay CE	10^{-4}	0.900	0.127–0.279

Does moving toward the MSE maps improve live decoding? Averaging each token-trained map with its MSE counterpart improves live throughput and acceptance length without retraining on both MATH and Code. All five maps change, so the test cannot identify one responsible stream. Appendix Table 22 reports every endpoint.

Does initialization explain the difference? To test whether Xavier initialization explains MSE’s advantage, we retrain from a rectangular identity, which copies the coordinates that fit the narrower interface and sets the rest to zero. All other MSE settings stay fixed, so its comparison isolates initialization. MSE acceptance length is nearly unchanged on MATH and GSM8K and slightly lower on Code and Chat, while its throughput falls on all four workloads (Table 12). AUF and decay CE improve both measurements on every workload but remain below MSE in acceptance length. Appendix Tables 24 and 25 report all endpoints.

Table 12: Initialization results on 128 prompts per task. Each cell shows Xavier → rectangular-identity initialization. Throughput is measured in tokens/s, and acceptance length τ is measured in tokens per verification step. For each dataset and metric, bold marks the best of all six values, and underline marks the second best.

Metric	Recipe	MATH	GSM8K	Code	Chat
Throughput (tokens/s)	MSE	218.75 → 213.70	172.35 → 166.70	125.98 → 119.49	78.31 → 76.13
	AUF	194.22 → 206.16	162.05 → 175.35	58.37 → 73.10	49.93 → 60.77
	Decay CE	194.69 → 203.74	163.47 → 169.72	61.56 → 76.02	51.31 → 61.11
Acceptance length τ (tokens/step)	MSE	7.973 → 7.983	6.204 → 6.217	4.690 → 4.569	2.802 → 2.747
	AUF	6.892 → 7.326	5.677 → 6.166	2.346 → 2.931	1.767 → 2.163
	Decay CE	6.947 → 7.348	5.693 → 6.073	2.449 → 3.042	1.812 → 2.183

7 CONCLUSION

A speculative drafter that reads its target’s hidden states becomes mismatched whenever that target is fine-tuned or replaced, and the usual remedy is to train a new drafter. We asked how much of that adaptation can be moved into the drafter’s input interface instead. RelaySpec trains one linear map per target feature stream, keeps the draft transformer, fusion projection, normalization, embeddings and output head frozen, and folds the maps into the existing projection so that inference runs the original architecture with one updated matrix.

The evidence supports a specific and bounded claim. For a fine-tuned target the maps improve every adapter we tested at two model scales, and the gain tracks acceptance length rather than any change to the decoding path. For a replaced target within a model family, reconstruction of the original interface recovers essentially all of a purpose-built native drafter on mathematical reasoning and most of it elsewhere, without any target-specific drafter training. Across families and tokenizers the same construction still accelerates the target but no longer matches a native drafter, which we

report as the boundary of the method rather than a gap to be explained away. The construction is not specific to DFlash: three maps on a frozen EAGLE-3 drafter reproduce both results, including the pairing of objective to setting.

Two findings explain why this works and are worth separating from the throughput numbers. The factorization into one map per stream is doing real work, since a single matrix of the same size, initialized to the same function, reaches far less under a matched objective. And the useful part of the learned correction is directional: the component perpendicular to the original fused context recovers nearly all of the benefit, while the parallel component leaves acceptance at the unadapted level, which is what RMSNorm predicts. The maps redirect a usable context rather than undoing the change the fine-tune introduced.

The practical consequence is that supporting a new target variant becomes a routine operation. Adaptation uses two orders of magnitude less data than the published recipe for training the drafter, is already effective from as few as 16 examples, adds 0.33 GiB of GPU memory, and leaves the draft transformer shared across targets. The limits we did not overcome are cross-tokenizer transfer, code and chat workloads under transfer, and high-concurrency serving where acceptance length stops being the bottleneck. Extending the interface to those settings, and to longer contexts and other accelerators, is the natural next step.

AI USE STATEMENT

We used large language model assistants for code support and for editing prose. Assistants helped write and debug experiment, evaluation and figure-generation scripts, and helped revise wording and structure in this manuscript. All research questions, experimental designs, model and data choices, and claims are the authors'. Every number reported here comes from our own recorded measurements, and the authors verified each claim against those records. The authors take full responsibility for the content of this paper, including any remaining errors.

ETHICS STATEMENT

This work studies inference efficiency for existing language models. It does not train or release a new target model, does not change any target's output distribution under greedy verification, and does not introduce new capabilities or new training data about people. All targets, drafters and datasets are publicly released artifacts used under their own licenses, and the domain adapters we evaluate come from previously released checkpoints.

The main societal consideration is indirect. Faster inference lowers the cost of deploying language models, which amplifies both their benefits and any harms of the underlying target model. RelaySpec inherits whatever behavior, bias or failure mode its target already has, and speculative decoding provides no additional safety property. Because adaptation is cheap, it also lowers the cost of maintaining many specialized target variants, which makes the provenance of each adapter worth tracking in a deployment. We report negative and partial results, including the workloads and model pairs where the method does not reach a native drafter, so that the method's limits are visible to anyone considering it.

REPRODUCIBILITY STATEMENT

Section 5.1 states the drafter and target checkpoints, adapter sources, feature layer indices, map shapes and trained parameter counts, the training data and schedules for both settings, and the decoding, hardware and backend configuration used for every measurement. Section 4 gives the exact objectives, including the accept-until-fail support mask and the relative reconstruction error with its normalization and epsilon. Every reported cell uses 128 held-out prompts, and the response caps are stated per workload.

The appendix reports the complete measurements behind every analysis claim in Section 6, including matching controls, all tested variants, negative results, metric definitions and units, and the timing repeats that bound run-to-run variation. Where a comparison comes from a separate cohort, a different architecture or a different learning rate, we state that alongside the numbers rather than presenting it as a matched ablation. Figures 3 and 4 are generated directly from the recorded mea-

surement tables by scripts released with the manuscript source, and the plotted values are saved alongside them.

Throughput measurements depend on hardware, backend version and batching, so absolute tokens per second should be expected to move across environments. The ratios we report are computed within a single environment against a control measured in that same environment, which is the comparison we intend to be reproducible.

REFERENCES

- Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost parallel draft model adaptation. In *International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=XbOyv7iVGL>.
- Zihao An, Taichi Liu, Ziqiong Liu, Dong Li, Ruofeng Liu, and Emad Barsoum. PARD-2: Target-aligned parallel draft model for dual-mode speculative decoding. *arXiv preprint arXiv:2605.08632*, 2026b. URL <https://arxiv.org/abs/2605.08632>.
- Frédéric Berdoz, Peer Rheinboldt, and Roger Wattenhofer. Steering pretrained drafters during speculative decoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL <https://ojs.aaai.org/index.php/AAAI/article/view/40255>.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 5209–5235. PMLR, 2024. URL <https://proceedings.mlr.press/v235/cai24b.html>.
- Sahil Chaudhary. Code Alpaca: An instruction-following LLaMA model for code generation. GitHub repository, <https://github.com/sahil280114/codealpaca>, 2023.
- Alan Chen, Jack Merullo, Alessandro Stolfo, and Ellie Pavlick. Transferring linear features across language models with model stitching. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.nips.cc/paper_files/paper/2025/hash/4569a868e7aa891248832ec08445d071-Abstract-Conference.html.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2602.06036>.
- Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, robust, and hardware-aware speculative decoding. *arXiv preprint arXiv:2402.12374*, 2024. URL <https://arxiv.org/abs/2402.12374>.
- Yunfei Cheng, Aonan Zhang, Xuanyu Zhang, Chong Wang, and Yi Wang. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024. URL <https://arxiv.org/abs/2403.09919>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. UltraFeedback: Boosting language models with scaled AI feedback. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2310.01377>.

- Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023. URL <https://arxiv.org/abs/2307.08691>.
- Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018. URL <https://arxiv.org/abs/1702.03118>.
- Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. URL <https://arxiv.org/abs/2404.16710>.
- Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of LLM inference using lookahead decoding. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2402.02057>.
- Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pp. 249–256. PMLR, 2010. URL <https://proceedings.mlr.press/v9/glorot10a.html>.
- Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- Youcheng Huang, Chen Huang, Duanyu Feng, Wenqiang Lei, and Jiancheng Lv. Cross-model transferability among large language models on the platonic representations of concepts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3686–3704. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.185. URL <https://aclanthology.org/2025.acl-long.185/>.
- Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. Repspec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554clf-Abstract-Conference.html.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023. URL <https://doi.org/10.1145/3600006.3613165>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.

- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath. Hugging Face repository, <https://huggingface.co/AI-MO/NuminaMath-CoT>, 2024a.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 28935–28948. PMLR, 2024b. URL <https://proceedings.mlr.press/v235/li24bt.html>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024c. URL <https://aclanthology.org/2024.emnlp-main.422/>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/c7b5a35ea98b62512a869c19ea7b03cb-Abstract-Conference.html.
- Luxi Lin, Zhihang Lin, Zhanpeng Zeng, Yuhao Chen, Qingyu Zhang, Jixiang Luo, Xuelong Li, and Rongrong Ji. Efficiently aligning draft models via parameter- and data-efficient adaptation. *arXiv preprint arXiv:2603.09527*, 2026. URL <https://arxiv.org/abs/2603.09527>.
- Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Online speculative decoding. *arXiv preprint arXiv:2310.07177*, 2024. URL <https://arxiv.org/abs/2310.07177>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024. URL <https://arxiv.org/abs/2305.09781>.
- Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. Omnidraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/3c2fe1417eed1c6ff9acf169617981ea-Paper-Conference.pdf.
- Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL <https://arxiv.org/abs/1811.03115>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://arxiv.org/abs/1706.03762>.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020. URL <https://arxiv.org/abs/1910.03771>.

- Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. URL <https://arxiv.org/abs/2401.07851>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Tianjian Yang and Meng Li. Spec-AUF: Accept-until-fail training under train-inference misalignment for masked block drafters. *arXiv preprint arXiv:2607.01893*, 2026. URL <https://arxiv.org/abs/2607.01893>.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>.
- Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. URL <https://arxiv.org/abs/2309.08168>.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2312.07104>.
- Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. Distillspec: Improving speculative decoding via knowledge distillation. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/8766fbc68e1cd712ce273e0a363-Abstract-Conference.html.

A COMPLETE ANALYSIS PROBE RESULTS

This appendix reports the complete results for every probe cited in Section 6. It also reports several completed interventions that are not discussed in the main text. All results come from the Phase 1 and Phase 2 experimental reports. They were checked against the corresponding Markdown probe reports while preparing this appendix.

A.1 LORA-FINE-TUNED TARGETS: DIRECTION AND SCALE

For each token position, let z_0 be the fused context without the learned maps and let δ be their learned correction. The dynamic intervention constructs $z_0 + \delta_{\parallel}$ or $z_0 + \delta_{\perp}$ before RMSNorm. Native and full dynamic controls use the same hook path. Every row in Table 13 uses the same 128 held-out prompts for its task.

The NanoCoder timing ratio above 100% is not evidence that the perpendicular component is functionally better than the full map. The full dynamic control saves only 9.218 seconds relative to its native dynamic control, while the perpendicular intervention saves 18.118 seconds in a separate measurement. The acceptance-length comparison is more stable and is the basis of the main-text interpretation.

The scaling probes use folded static projections. Correction strength α supplies $z_0 + \alpha\delta$. Whole-context scaling instead supplies $2(z_0 + \delta)$. Table 14 includes the native and full endpoints, every tested correction strength, and the whole-context scaling control cited in Section 6.1.

Whole-context doubling preserves the recorded acceptance length exactly on both tasks. Correction doubling does not. Relative to the full maps, mean throughput changes by +0.69% on GSM8K and -3.14% on NanoCoder. Acceptance length is a pooled ratio and does not determine each request’s

Table 13: Complete direction intervention results. Mean throughput is measured in tokens/s. Acceptance length τ is measured in tokens per verification step and includes the target contribution. Summed time is measured in seconds over 128 prompts. Gain and retained latency reduction are percentages relative to the matching dynamic controls.

Task	Context	Mean throughput	Gain	τ	Summed time	Retained latency reduction
GSM8K	Native z_0	131.47	0.00%	4.472	149.636	0.0%
GSM8K	Full $z_0 + \delta$	180.17	37.05%	5.988	115.310	100.0%
GSM8K	Parallel $z_0 + \delta_{\parallel}$	124.61	-5.21%	4.466	157.729	-23.6%
GSM8K	Perpendicular $z_0 + \delta_{\perp}$	177.64	35.12%	5.924	117.436	93.8%
NanoCoder	Native z_0	150.58	0.00%	5.111	191.998	0.0%
NanoCoder	Full $z_0 + \delta$	161.63	7.33%	5.552	182.780	100.0%
NanoCoder	Parallel $z_0 + \delta_{\parallel}$	150.78	0.13%	5.114	191.847	1.6%
NanoCoder	Perpendicular $z_0 + \delta_{\perp}$	170.39	13.15%	5.527	173.880	196.6%

Table 14: Complete correction-strength and whole-context scaling results. Mean throughput is measured in tokens/s. Acceptance length τ is measured in tokens per verification step. Gain and retained latency reduction are percentages relative to the matching static native and full controls.

Task	Static intervention	Mean throughput	Gain	τ	Retained latency reduction
GSM8K	Native, $\alpha = 0$	128.90	0.00%	4.472	0.0%
GSM8K	Correction strength 0.5	177.02	37.33%	5.757	98.9%
GSM8K	Full, $\alpha = 1$	179.83	39.51%	5.988	100.0%
GSM8K	Correction strength 1.5	175.61	36.24%	5.802	89.8%
GSM8K	Correction strength 2	165.93	28.72%	5.400	72.8%
GSM8K	Whole context $\times 2$	181.07	40.47%	5.988	102.1%
NanoCoder	Native, $\alpha = 0$	150.79	0.00%	5.111	0.0%
NanoCoder	Correction strength 0.5	169.23	12.23%	5.532	90.8%
NanoCoder	Full, $\alpha = 1$	171.73	13.89%	5.552	100.0%
NanoCoder	Correction strength 1.5	163.65	8.53%	5.321	54.0%
NanoCoder	Correction strength 2	152.46	1.11%	5.032	-4.4%
NanoCoder	Whole context $\times 2$	166.34	10.32%	5.552	72.4%

block schedule, output length, or token path. The variants were timed in separate, unrepeated passes, while scaling adds no inference operation. We therefore treat these small throughput differences as likely GPU timing variation rather than a causal speed effect.

A.2 LORA-SHIFT GEOMETRY

The geometry probe teacher-forces identical saved tokens through the base target and the LoRA-fine-tuned target. It uses 64 cached training examples per task and 16 response positions per example, for 1,024 positions. Let t be the fused-context change caused by LoRA fine-tuning and let δ be the map correction. The shuffled control randomly reassigns t among sampled positions while leaving every correction fixed.

Table 15: Complete LoRA-shift geometry results. Cosine similarity, radial energy fraction, and relative linearization error are dimensionless.

Task	Positions	Mean cosine	Shuffled cosine	Difference	Radial energy	RMSNorm error
GSM8K	1,024	-0.0429	-0.0413	-0.0016	25.22%	17.55%
NanoCoder	1,024	0.1248	0.1052	0.0195	34.82%	14.88%

Radial energy is the fraction of $\sum_p \|\delta_p\|^2$ parallel to $z_{0,p}$. RMSNorm error is the norm of the difference between the full normalized change and its first-order approximation, divided by the norm of the full normalized change. Neither quantity is an acceptance or speed measurement.

A.3 QWEN3-4B DRAFTER TO QWEN3-8B TARGET: FEATURE RECONSTRUCTION

The feature probe uses 64 training records and 7,090 sampled positions, split into 2,994 prompt and 4,096 response positions. Feature and context errors are normalized squared errors and are dimensionless. Context error is measured after frozen fusion and RMSNorm. Source cosine is the dimensionless cosine similarity between a mapped 8B feature and its paired 4B feature. The MSE, AUF, and decay rows are complete training recipes with different learning rates and supervision coverage.

Table 16: Complete fused-context reconstruction results. Errors are dimensionless normalized squared errors. Lower is better.

Recipe	Prompt context error	Response context error
MSE	0.216170	0.136147
AUF	1.228544	0.940413
Decay CE	1.189191	0.899731

Table 17: Complete per-layer feature reconstruction results. Feature error is a dimensionless normalized squared error, and source cosine is dimensionless. P and R denote prompt and response positions. Layer numbers index the five feature streams in RelaySpec.

MSE					AUF				
Layer	P error	R error	P cosine	R cosine	Layer	P error	R error	P cosine	R cosine
0	0.060359	0.033437	0.975595	0.984793	0	2.742802	2.794326	0.195425	0.214868
1	0.191632	0.068966	0.899016	0.965129	1	2.859036	2.540209	0.112855	0.120631
2	0.192591	0.083779	0.901045	0.957394	2	21.592222	12.066430	0.141729	0.175011
3	0.191474	0.110141	0.902611	0.944060	3	6.882195	4.716092	0.144134	0.226237
4	0.180501	0.090359	0.909023	0.954783	4	3.814032	3.408715	0.190191	0.266906

Decay CE				
Layer	P error	R error	P cosine	R cosine
0	2.710681	2.753600	0.201154	0.220411
1	2.827380	2.522013	0.119284	0.126873
2	21.544376	12.034999	0.145896	0.179880
3	6.830830	4.675968	0.151039	0.234286
4	3.730459	3.298069	0.201127	0.278744

A.4 HELD-OUT COMMON-INPUT DIAGNOSTICS

For each workload, the probe selects eight held-out responses and eight identical test positions per response, for 64 tests per variant. At each position, the frozen drafter predicts the next 15 tokens from the saved response. Full cross-entropy is measured in nats per predicted token. AUF support is the percentage of valid tokens that would receive AUF loss. Correct prefix is the mean number of consecutive correct predictions before the first error and is measured in tokens. Prompt and response errors are dimensionless normalized context errors. These offline prefixes are not live acceptance lengths.

Table 18: Complete held-out MATH diagnostics. Cross-entropy is in nats/token, AUF support is a percentage, correct prefix is in tokens, and context errors are dimensionless.

Variant	Cross-entropy	AUF support	Correct prefix	Prompt error	Response error
MSE	1.6939	51.81%	6.7656	0.1821	0.1236
Decay midpoint	1.8043	49.79%	6.4844	0.7590	0.4928
Decay CE	1.9534	47.13%	6.0625	1.3852	0.9058
Warm unseen 4k	2.0634	52.45%	6.9219	0.3910	0.3867
Warm seen 4k	2.0695	50.64%	6.6094	0.3947	0.3827
AUF midpoint	2.1282	48.51%	6.2812	0.7768	0.5110
Warm unseen 8k	2.1507	54.47%	7.2188	0.4347	0.4084
Warm seen 8k	2.1760	56.49%	7.5625	0.4362	0.4088
AUF	2.6028	45.74%	5.8438	1.4131	0.9333
Random initial	8.6261	6.81%	0.0000	4.4041	4.9689

Table 19: Complete held-out GSM8K diagnostics. Units match Table 18.

Variant	Cross-entropy	AUF support	Correct prefix	Prompt error	Response error
Decay midpoint	1.5407	55.01%	7.1875	0.7967	0.4933
Decay CE	1.6564	50.70%	6.5312	1.4673	0.8850
Warm seen 4k	1.7356	50.91%	6.5625	0.4074	0.3686
Warm seen 8k	1.7586	57.48%	7.5781	0.4506	0.3911
Warm unseen 8k	1.7793	55.65%	7.3125	0.4535	0.3911
MSE	1.7817	53.07%	6.8750	0.2026	0.1146
Warm unseen 4k	1.8745	55.65%	7.2812	0.4053	0.3692
AUF midpoint	1.9572	55.76%	7.3125	0.8212	0.5098
AUF	2.0566	54.14%	7.0781	1.5231	0.9074
Random initial	9.6898	7.00%	0.0156	4.0335	4.6817

Table 20: Complete held-out Code diagnostics. Units match Table 18.

Variant	Cross-entropy	AUF support	Correct prefix	Prompt error	Response error
MSE	2.6039	40.46%	4.8750	0.3942	0.2864
Warm unseen 4k	3.4776	32.24%	3.6719	0.5446	0.4510
Warm unseen 8k	3.5000	32.79%	3.7344	0.5785	0.4877
Warm seen 8k	3.5864	32.02%	3.6250	0.5754	0.4818
Warm seen 4k	3.6100	33.77%	3.8906	0.5456	0.4543
Decay midpoint	3.7045	25.33%	2.6406	0.9133	0.6970
AUF midpoint	4.1200	28.62%	3.1250	0.9301	0.7159
Decay CE	4.6468	17.98%	1.5625	1.5436	1.2464
AUF	5.5081	17.00%	1.4219	1.5820	1.2868
Random initial	9.8854	7.24%	0.0312	3.7889	4.6308

Table 21: Complete held-out Chat diagnostics. Units match Table 18.

Variant	Cross-entropy	AUF support	Correct prefix	Prompt error	Response error
MSE	3.8074	28.50%	2.9062	0.3017	0.3255
Warm unseen 8k	4.1924	26.64%	2.6406	0.4872	0.4895
Warm seen 4k	4.2015	26.99%	2.6719	0.4507	0.4547
Warm seen 8k	4.2630	24.65%	2.3594	0.4868	0.4873
Warm unseen 4k	4.2904	24.42%	2.3281	0.4504	0.4519
Decay midpoint	4.5984	21.03%	1.8750	0.9437	0.7632
AUF midpoint	4.9769	22.78%	2.1094	0.9668	0.7841
Decay CE	5.6398	16.36%	1.2188	1.7617	1.3749
AUF	6.2094	13.55%	0.8438	1.8059	1.4210
Random initial	10.7415	7.48%	0.0000	4.9159	4.0375

A.5 COMPLETE LIVE MIDPOINT INTERVENTION

For each token-trained parent, the midpoint averages all five parent maps with the corresponding MSE maps. There is no retraining. Each endpoint uses 128 prompts. Mean and pooled throughput are measured in tokens/s. Acceptance length is measured in tokens per verification step. Total time is measured in seconds. Native and parent comparisons are shown with multiplication signs. Latency reduction is a percentage relative to the named cold parent. Endpoints were timed in separate passes.

Table 22: All parent, midpoint, and MSE endpoints in the live interpolation probe.

Task	Parent	Variant	Mean throughput	Pooled throughput	Native ratio	Parent ratio	Total time	Latency reduction	τ
Code	AUF	MSE	126.18	124.01	0.8322×	2.1618×	569.96	50.08%	4.6912
Code	AUF	Midpoint	81.64	85.41	0.5384×	1.3986×	827.50	27.53%	3.2512
Code	AUF	Parent	58.37	61.90	0.3850×	1.0000×	1141.85	0.00%	2.3456
Code	Decay CE	MSE	126.18	124.01	0.8322×	2.0498×	569.96	47.75%	4.6912
Code	Decay CE	Midpoint	84.68	88.03	0.5585×	1.3756×	802.88	26.39%	3.3368
Code	Decay CE	Parent	61.56	64.80	0.4060×	1.0000×	1090.76	0.00%	2.4489
MATH	AUF	MSE	219.48	216.72	0.9986×	1.1300×	406.20	13.83%	7.9776
MATH	AUF	Midpoint	206.81	201.87	0.9410×	1.0648×	436.09	7.48%	7.5624
MATH	AUF	Parent	194.22	186.76	0.8837×	1.0000×	471.37	0.00%	6.8923
MATH	Decay CE	MSE	219.48	216.72	0.9986×	1.1273×	406.20	13.39%	7.9776
MATH	Decay CE	Midpoint	208.80	203.73	0.9500×	1.0725×	432.11	7.86%	7.5748
MATH	Decay CE	Parent	194.69	187.71	0.8858×	1.0000×	468.98	0.00%	6.9474

A.6 COMPLETE WARM AUF REFINEMENT RESULTS

All four refinements start from the same 8k MSE parent and run AUF for three epochs. Seen data overlap the MSE training examples. Unseen data are disjoint from them. Mean throughput is measured in tokens/s. Acceptance length is measured in tokens per verification step. Fit time is measured in minutes and includes the MSE parent and AUF stage.

Table 23: All 16 warm-refinement endpoints. Native and MSE-parent throughput ratios use multiplication signs. Latency reduction is a percentage relative to native.

Task	AUF data	Native ratio	MSE-parent ratio	Mean throughput	Latency reduction	τ	Fit time
Chat	Unseen 4k	0.8548×	0.9624×	76.51	−17.33%	2.7495	32.39
Chat	Seen 8k	0.8446×	0.9509×	75.59	−19.06%	2.7038	45.32
Chat	Unseen 8k	0.8431×	0.9492×	75.46	−19.32%	2.7030	47.42
Chat	Seen 4k	0.8420×	0.9480×	75.36	−18.58%	2.7403	31.52
Code	Unseen 4k	0.7417×	0.8913×	112.46	−26.52%	4.3499	32.39
Code	Seen 4k	0.7273×	0.8740×	110.28	−27.86%	4.2769	31.52
Code	Unseen 8k	0.7051×	0.8473×	106.92	−31.27%	4.1818	47.42
Code	Seen 8k	0.7043×	0.8463×	106.78	−32.01%	4.1525	45.32
GSM8K	Unseen 8k	1.0606×	1.0874×	188.72	5.53%	6.6218	47.42
GSM8K	Seen 8k	1.0431×	1.0696×	185.62	3.50%	6.5891	45.32
GSM8K	Unseen 4k	1.0413×	1.0677×	185.30	4.01%	6.5340	32.39
GSM8K	Seen 4k	1.0339×	1.0601×	183.97	3.40%	6.4951	31.52
MATH	Seen 8k	0.9927×	0.9941×	218.18	−1.32%	7.8662	45.32
MATH	Unseen 8k	0.9911×	0.9925×	217.82	−1.83%	7.8410	47.42
MATH	Seen 4k	0.9717×	0.9731×	213.57	−3.20%	7.8145	31.52
MATH	Unseen 4k	0.9694×	0.9707×	213.05	−3.42%	7.7785	32.39

The warm runs improve GSM8K and weaken MATH, Code, and Chat relative to the MSE parent. This pattern is consistent with workload-specific refinement, but it does not establish overfitting because matched training and held-out learning curves were not recorded.

A.7 RECTANGULAR-IDENTITY INITIALIZATION

This probe repeats the T1 8k MSE, AUF, and decaying-cross-entropy fits from a rectangular identity. Each map initially copies the first 2,560 coordinates of the 4,096-dimensional 8B feature and sets the remaining input directions to zero. Corresponding coordinates need not have the same meaning in the two models. All three retries use 8,192 training examples, three epochs, and learning rate 10^{-3} . The fusion projection, normalization, embeddings, vocabulary head, and drafter transformer remain frozen.

For MSE, the original and retry both use learning rate 10^{-3} , so their only planned recipe change is map initialization. The original AUF and decay fits use Xavier initialization and learning rate 10^{-4} . Their retries change both initialization and learning rate. Their improvements cannot be assigned to either change alone.

Table 24: Complete rectangular-identity endpoints. Mean throughput is measured in tokens/s. Acceptance length τ is measured in tokens per verification step. Native throughput ratios are shown with multiplication signs. Each workload uses 128 prompts.

Workload	Recipe	Mean throughput	Native ratio	τ
MATH	MSE	213.70	0.9723×	7.983
MATH	AUF	206.16	0.9380×	7.326
MATH	Decay CE	203.74	0.9270×	7.348
GSM8K	MSE	166.70	0.9368×	6.217
GSM8K	AUF	175.35	0.9854×	6.166
GSM8K	Decay CE	169.72	0.9538×	6.073
Code	MSE	119.49	0.7881×	4.569
Code	AUF	73.10	0.4821×	2.931
Code	Decay CE	76.02	0.5014×	3.042
Chat	MSE	76.13	0.8506×	2.747
Chat	AUF	60.77	0.6790×	2.163
Chat	Decay CE	61.11	0.6828×	2.183

Table 25: Complete comparison with the original Xavier-initialized fits. Throughput columns are measured in tokens/s. Acceptance length columns are measured in tokens per verification step. Changes are percentages relative to the original fit.

Workload	Recipe	Original throughput	Identity throughput	Change	Original τ	Identity τ	Change
MATH	MSE	218.75	213.70	-2.31%	7.973	7.983	+0.14%
MATH	AUF	194.22	206.16	+6.15%	6.892	7.326	+6.29%
MATH	Decay CE	194.69	203.74	+4.65%	6.947	7.348	+5.77%
GSM8K	MSE	172.35	166.70	-3.28%	6.204	6.217	+0.21%
GSM8K	AUF	162.05	175.35	+8.21%	5.677	6.166	+8.60%
GSM8K	Decay CE	163.47	169.72	+3.82%	5.693	6.073	+6.68%
Code	MSE	125.98	119.49	-5.15%	4.690	4.569	-2.58%
Code	AUF	58.37	73.10	+25.24%	2.346	2.931	+24.94%
Code	Decay CE	61.56	76.02	+23.49%	2.449	3.042	+24.20%
Chat	MSE	78.31	76.13	-2.78%	2.802	2.747	-1.94%
Chat	AUF	49.93	60.77	+21.73%	1.767	2.163	+22.41%
Chat	Decay CE	51.31	61.11	+19.09%	1.812	2.183	+20.52%

Table 26: Rectangular-identity fitting cost. Wall time is measured in minutes. GPU time is measured in GPU-hours on two GPUs.

Recipe	Wall time	GPU time
MSE	11.11	0.370
AUF	30.14	1.005
Decay CE	29.54	0.985

MSE retains the highest acceptance length on all four workloads and the highest measured throughput on MATH, Code, and Chat. AUF is faster on GSM8K in this single pass despite having a slightly lower acceptance length. There are no repeated timing trials, so this disagreement does not establish a stable throughput advantage or timing noise. The MSE initializer-only comparison supports retaining the selected Xavier initialization. The token-recipe retries show that their original initialization and learning rate are not the only settings under which they can learn, but do not isolate which change helped.

A.8 ADDITIONAL COMPLETED PROBES NOT USED IN THE MAIN ANALYSIS

The following completed probes informed the decision to keep the main analysis narrow. They are reported here so negative and secondary results remain visible.

A.8.1 REMOVING ONE MAP CORRECTION

Removing map i replaces W_i with the identity and leaves its native feature stream and fusion block in place. Map indices 0 through 4 correspond to target layers 1, 9, 17, 25, and 33. Throughput is measured in tokens/s. Acceptance length is measured in tokens per verification step.

Table 27: Complete single-map removal results. Gain and retained latency reduction are percentages relative to the static controls.

Task	Removed map	Mean throughput	Gain	τ	Retained latency reduction
GSM8K	0	174.69	35.52%	5.982	88.6%
GSM8K	1	178.72	38.65%	5.979	97.9%
GSM8K	2	184.68	43.27%	6.006	109.6%
GSM8K	3	176.18	36.68%	5.908	94.9%
GSM8K	4	171.68	33.18%	5.833	86.4%
NanoCoder	0	167.18	10.87%	5.577	74.9%
NanoCoder	1	170.44	13.03%	5.569	90.6%
NanoCoder	2	170.78	13.26%	5.587	92.9%
NanoCoder	3	170.51	13.08%	5.539	92.8%
NanoCoder	4	168.31	11.62%	5.447	84.9%

A.8.2 USING MAPS TRAINED FOR ANOTHER LORA-FINE-TUNED TARGET

The evaluated target, target LoRA, and prompts remain fixed while the complete five-map bank is replaced. No completed KiCad destination comparison is available.

Table 28: Complete foreign-bank results. Throughput is measured in tokens/s. Acceptance length τ is measured in tokens per verification step. Gain and retained latency reduction are percentages relative to the destination task’s native and full controls.

Destination	Source bank	Mean throughput	Gain	τ	Retained latency reduction
GSM8K	KiCad	127.63	−0.99%	4.430	−2.5%
GSM8K	NanoCoder	129.72	0.63%	4.565	4.2%
NanoCoder	GSM8K	148.97	−1.20%	5.016	−2.0%
NanoCoder	KiCad	148.12	−1.77%	4.990	−11.1%

A.8.3 LOW-RANK CORRECTION PROBES

Whole-map truncation applies SVD to W_i and removes most of the identity path. Correction truncation applies SVD only to $W_i - I$ and then restores the identity. Every reported rank is per map. All exports still use a dense folded projection, so these measurements do not demonstrate a low-rank inference-speed benefit.

Table 29: Complete first-pass low-rank results. Throughput is measured in tokens/s. Acceptance length τ is measured in tokens per verification step. Gain and retained latency reduction are percentages relative to static controls.

Task	Truncated object	Rank	Mean throughput	Gain	τ	Retained latency reduction
GSM8K	Whole map	1	34.16	-73.50%	1.145	-1142.5%
GSM8K	Whole map	2	34.50	-73.24%	1.165	-1125.0%
GSM8K	Correction	1	143.26	11.14%	4.913	36.9%
GSM8K	Correction	2	148.99	15.58%	5.072	49.6%
GSM8K	Correction	64	172.81	34.06%	5.931	87.6%
GSM8K	Correction	128	179.10	38.94%	5.928	99.2%
GSM8K	Correction	256	172.59	33.89%	5.964	87.0%
NanoCoder	Whole map	1	31.06	-79.40%	1.057	-3484.4%
NanoCoder	Whole map	2	33.06	-78.08%	1.076	-3248.1%
NanoCoder	Correction	1	157.11	4.19%	5.239	38.8%
NanoCoder	Correction	2	151.77	0.65%	5.259	2.1%
NanoCoder	Correction	128	163.60	8.50%	5.492	60.1%
NanoCoder	Correction	256	168.04	11.44%	5.521	83.1%
NanoCoder	Correction	512	167.20	10.89%	5.528	76.2%
NanoCoder	Correction	1,024	168.84	11.97%	5.546	84.5%
NanoCoder	Correction	2,048	166.49	10.41%	5.553	72.3%

Table 30: Complete repeated timing results for selected correction ranks. Throughput is measured in tokens/s. Acceptance length τ is measured in tokens per verification step. Gain and retained latency reduction are percentages relative to the native and full controls from the same repeat.

Task and repeat	Variant	Mean throughput	Gain	τ	Retained latency reduction
GSM8K, repeat 0	Native	124.91	0.00%	4.472	0.0%
GSM8K, repeat 0	Full	176.03	40.92%	5.988	100.0%
GSM8K, repeat 0	Correction rank 128	172.81	38.35%	5.928	95.1%
GSM8K, repeat 0	Correction rank 256	180.10	44.18%	5.964	106.7%
GSM8K, repeat 1	Native	127.34	0.00%	4.472	0.0%
GSM8K, repeat 1	Full	184.00	44.50%	5.988	100.0%
GSM8K, repeat 1	Correction rank 128	183.07	43.76%	5.928	99.6%
GSM8K, repeat 1	Correction rank 256	176.10	38.29%	5.964	88.6%
NanoCoder, repeat 0	Native	152.44	0.00%	5.111	0.0%
NanoCoder, repeat 0	Full	171.03	12.20%	5.552	100.0%
NanoCoder, repeat 0	Correction rank 256	163.49	7.25%	5.521	57.8%
NanoCoder, repeat 0	Correction rank 512	166.52	9.24%	5.528	72.4%
NanoCoder, repeat 1	Native	157.04	0.00%	5.111	0.0%
NanoCoder, repeat 1	Full	172.93	10.12%	5.552	100.0%
NanoCoder, repeat 1	Correction rank 256	171.45	9.18%	5.521	89.3%
NanoCoder, repeat 1	Correction rank 512	166.80	6.22%	5.528	52.1%